Experiment 3

Database Schema for a customer-sale scenario
Customer(Cust id : integer, cust_name: string)
Item(item_id: integer, item_name: string, price: integer)
Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)
For the above schema, perform the following—

1. Create the tables with the appropriate integrity constraints.

```
CREATE DATABASE IF NOT EXISTS CustomerSaleDB;
USE CustomerSaleDB;

CREATE TABLE Customer (
   cust_id INT PRIMARY KEY,
   cust_name VARCHAR(100) NOT NULL
);


CREATE TABLE Item (
   item_id INT PRIMARY KEY,
   item_name VARCHAR(100) NOT NULL,
   price INT NOT NULL CHECK (price > 0)
);

CREATE TABLE Sale (
   bill_no INT,
   bill_date DATE NOT NULL,
   cust_id INT,
   item_id INT,
   qty_sold INT NOT NULL CHECK (qty_sold > 0),
   PRIMARY KEY (bill_no, item_id),
   FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
   FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
```

	cust_id	cust_name
*	NULL	NULL

	item_id	item_name	price
*	NULL	NULL	NULL

	bill_no	bill_date	cust_id	item_id	qty_sold
*	NULL	NULL	NULL	NULL	NULL

2. Insert around 10 records in each of the tables.

bill_no	cust_name	item_id
1001	Ishaan	101
1002	Mahin	102
1003	Aditya	104
1004	Ayush	105
1005	Priya	108
1006	Karan	110
1007	Neha	103
1008	Rohit	109
1009	Simran	106
1010	Vikas	107

```
INSERT INTO Customer VALUES
(1,'Ishaan'),
(2,'Mahin'),
(3,'Aditya'),
(4,'Ayush'),
(5,'Priya'),
(6,'Karan'),
(7,'Neha'),
(8,'Rohit'),
(9,'Simran'),
(10,'Vikas');

INSERT INTO Item VALUES
(101,'Keyboard',50000),
(102,'Mouse',500),
(103,'Keyboard',800),
(104,'Monitor',12000),
(105,'USB Cable',200),
(106,'Headphones',1500),
(107,'Charger',900),
(108,'Tablet',25000),
(109,'Smartphone',30000),
(110,'Pen Drive',700);

INSERT INTO Sale VALUES
(1001,CURDATE(),1,101,1),
(1002,CURDATE(),2,102,2),
(1003,CURDATE(),3,104,1),
(1004,CURDATE(),4,105,3),
(1005,CURDATE(),5,108,1),
(1006,CURDATE(),6,110,4),
(1007,CURDATE(),7,103,2),
(1008,CURDATE(),8,109,1),
(1009,CURDATE(),9,106,1),
(1010,CURDATE(),10,107,2);
```

cust_id	cust_name
1	Ishaan
2	Mahin
3	Aditya
4	Ayush
5	Priya
6	Karan
7	Neha
8	Rohit
9	Simran
10	Vikas
NULL	NULL

item_id	item_name	price
101	Keyboard	50000
102	Mouse	500
103	Keyboard	800
104	Monitor	12000
105	USB Cable	200
106	Headphones	1500
107	Charger	900
108	Tablet	25000
109	Smartphone	30000
110	Pen Drive	700
NULL	NULL	NULL

3. List all the bills for the current date with the customer names and item numbers.

```
SELECT s.bill_no, c.cust_name, s.item_id
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
WHERE s.bill_date = CURDATE();
```

bill_no	bill_date	cust_id	item_id	qty_sold
1001	2026-02-28	1	101	1
1002	2026-02-28	2	102	2
1003	2026-02-28	3	104	1
1004	2026-02-28	4	105	3
1005	2026-02-28	5	108	1
1006	2026-02-28	6	110	4
1007	2026-02-28	7	103	2
1008	2026-02-28	8	109	1
1009	2026-02-28	9	106	1
1010	2026-02-28	10	107	2
NULL	NULL	NULL	NULL	NULL

4. List the total Bill details with the quantity sold, price of the item and the final amount.

```
SELECT
   s.bill_no,
   s.qty_sold,
   i.price,
   (s.qty_sold * i.price) AS final_amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
```

bill_no	qty_sold	price	final_amount
1001	1	50000	50000
1002	2	500	1000
1003	1	12000	12000
1004	3	200	600
1005	1	25000	25000
1006	4	700	2800
1007	2	800	1600
1008	1	30000	30000
1009	1	1500	1500
1010	2	900	1800

5. List the details of the customer who have bought a product which has a price>200.

```
SELECT DISTINCT c.cust_id, c.cust_name
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE i.price > 200;
```

cust_id	cust_name
1	Ishaan
2	Mahin
7	Neha
3	Aditya
9	Simran
10	Vikas
5	Priya
8	Rohit
6	Karan

6. Give a count of how many products have been bought by each customer

```
SELECT
   c.cust_id,
   c.cust_name,
   SUM(s.qty_sold) AS total_products_bought
FROM Customer c
JOIN Sale s ON c.cust_id = s.cust_id
GROUP BY c.cust_id, c.cust_name;
```

	cust_id	cust_name	total_products_bought
▶	1	Ishaan	1
	2	Mahin	2
	3	Aditya	1
	4	Ayush	3
	5	Priya	1
	6	Karan	4
	7	Neha	2
	8	Rohit	1
	9	Simran	1
	10	Vikas	2

7. Give a list of products bought by a customer having cust_id as 5.

```
SELECT i.item_id, i.item_name
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.cust_id = 5;
```

	item_id	item_name
▶	108	Tablet

8. List the item details which are sold as of today.

```
SELECT DISTINCT i.*
FROM Item i
JOIN Sale s ON i.item_id = s.item_id
WHERE s.bill_date = CURDATE();
```

	item_id	item_name	price
▶	101	Keyboard	50000
	102	Mouse	500
	104	Monitor	12000
	105	USB Cable	200
	108	Tablet	25000
	110	Pen Drive	700
	103	Keyboard	800
	109	Smartphone	30000
	106	Headphones	1500
	107	Charger	900

9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.

```
CREATE OR REPLACE VIEW Bill_View AS
SELECT
    s.bill_no,
    s.bill_date,
```

```
    s.cust_id,
    s.item_id,
    i.price,
    s.qty_sold,
    (s.qty_sold * i.price) AS amount
FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
SELECT * FROM Bill_View;
```

bill_no	bill_date	cust_id	item_id	price	qty_sold	amount
1001	2026-02-28	1	101	50000	1	50000
1002	2026-02-28	2	102	500	2	1000
1003	2026-02-28	3	104	12000	1	12000
1004	2026-02-28	4	105	200	3	600
1005	2026-02-28	5	108	25000	1	25000
1006	2026-02-28	6	110	700	4	2800
1007	2026-02-28	7	103	800	2	1600
1008	2026-02-28	8	109	30000	1	30000
1009	2026-02-28	9	106	1500	1	1500
1010	2026-02-28	10	107	900	2	1800

10. Create a view which lists the daily sales date wise for the last one week

```
CREATE OR REPLACE VIEW Weekly_Sales AS
SELECT
    s.bill_date,
    SUM(s.qty_sold * i.price) AS total_sales
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.bill_date >= CURDATE() - INTERVAL 7 DAY
GROUP BY s.bill_date;
SELECT * FROM Weekly_Sales;
```

Bill_date:	2026-02-28
Total_sales:	126300